1. La forme d'un algorithme

```
FONCTION : nom (E x : entier, E T : tableau d'entiers) :
booléen
SPÉCIFICATIONS : ce qu'on suppose, ce qu'on renvoie.
Variable : cpt : entier
DÉBUT
    cpt ← 0
    pour i de 0 à taille(T)-1 faire
        si T[i] = x alors
            cpt ← cpt + 1
        fin si;
    fin pour;
    Renvoyer cpt > 0
FIN
```

- **FONCTION** : renvoie, type du résultat après la parenthèse. **PROCÉDURE** : ne renvoie rien (affiche, ou modifie un paramètre ES), pas de type de résultat.
- Paramètres : **E** entrée (lu), **S** sortie, **ES** entrée-sortie (modifié). On peut écrire « E D1, D2 : dictionnaires ».
- **Variable** : les mémoires de travail. **Pas** l'indice d'un pour, **pas** les paramètres (cours p. 11). Le compteur d'un tant que, **oui**.
- Affectation multiple : `cpt, M ← 0, L`. Initialiser avant d'utiliser.
- **Renvoyer** après la boucle (sauf recherche : « dès que je trouve »). Un Renvoyer dans une boucle arrête tout.

```
si a alors … sinon si b alors … sinon … fin si;
pour i de a à b [par pas de c] faire … fin pour;
tant que condition faire … fin tq;
```

2. Expressions, types, cond

- `/` division → nombre (5/2 = 2,5) · `div` quotient (5 div 2 = 2) · `mod` reste (5 mod 2 = 1) · `^` puissance.
- Comparaisons = ≠ < ≤ > ≥ → booléen. Booléens : **true**, **false**. Logique : non, et, ou, **paresseux** : « false et … » = false, « true ou … » = true sans calculer la suite.
- `cond(a, b, c)` = b si a vaut true, c sinon ; **b et c du même type**.
- « erreur » si : mauvais nombre d'arguments, booléen dans un calcul (3 * true), entier dans et/ou (reste(3) et reste(5)), cond de deux types.

TD 1 ex. 1 (proche?(a,b) : |a−b| < 2 ; reste(x) : x mod 2) : (2 + (3 * (7 − 5))) = **8** · 2 + (3 * true) **erreur** · (2 < 3) et false = **false** · proche?(1, 2, 3) **erreur** · proche?(1, 2) ou … = **true** · (reste(3) et reste(5)) < 3 **erreur** · proche?(reste(5), 2) = **true** · cond(proche?(4, 2), 2/0, 1) = **1** (2/0 jamais calculé) · cond(proche?(1, 4), false, proche?(2, 3)) = **true** · cond(1 < 3, reste(3), true) **erreur**.

Cours p. 13 : 1 + 5/2 = 3,5 · 8 div 3 + 7 mod 5 = 4 · true ou (5/0 = 1) = true · cond(5 div 2 = 1, true, 0) erreur · cond(5 div 2 = 2, cond(5 mod 2 = 0, 3, 8), 11) = 8.

max3 : `cond(a ≥ b et a ≥ c, a, cond(b ≥ c, b, c))` ou `max(a, max(b, c))`. ouExcl : `cond(a, non b, b)` ou `(a et non b) ou (non a et b)`.

3. Boucles et trace

- **pour** i de a à b : a et b compris ; pas de −2 : « pour j de 6 à 3 par pas de −2 » fait 6, 4.
- **tant que** : quelque chose doit changer dedans. `i ← 5 ; tant que i ≠ 0 : i ← i − 2` ne s'arrête jamais.
- **Trace** : un tableau, une colonne par variable, une ligne par tour + la ligne de départ ; écrire le test d'arrêt et la valeur renvoyée.
- Deux « si … fin si; » à la suite ≠ « si … sinon si … » (cours p. 15 : 7 et non 5).

i ← 0 ; tant que 2*i+1 < 10 : i ← i+1 → renvoie **5**. pour i de 1 à 5 : afficher 10−2*i → 8 6 4 2 0. nbChiffres(4097) : m = 4097, 409, 40, 4 ; c = 1, 2, 3, 4 → 4.

4. Motifs d'étoiles (affEtoile, affEspace, affRL)

Ligne i : combien d'espaces, combien d'étoiles (en fonction de i, n) ; une boucle par morceau ; **affRL() à la fin de chaque ligne**, dans la boucle des lignes.

carré : n × ligne(n) · triangle : ligne(i) · triangle inversé : i−1 espaces + n−i+1 étoiles · pyramide : n−i espaces + i fois « * » · Λ GrandEt (k de 0 à n−1) : n−1−k espaces, X, si k > 0 : 2k−1 espaces, X.

```
PROCÉDURE : pyramide (E n : entier)
DÉBUT
    pour i de 1 à n faire
        pour j de 1 à n-i faire affEspace() fin pour;
        pour j de 1 à i faire
            affEtoile()
            affEspace()
        fin pour;
        affRL()
    fin pour;
FIN
```

5. Récursivité : écrire

1. cas de base (sans appel) · **2.** récurrence : f(n) avec f(n−1) (ou queue(L)…) · **3.** cond ou si. Sans cas de base, ou si l'appel ne se rapproche pas du cas de base : appels infinis.

```
FONCTION : FactRec (E n : entier) : entier
SPÉCIFICATIONS : n ∈ ℕ. Renvoie n!.
DÉBUT
    si n = 0 alors
        Renvoyer 1
    sinon
        Renvoyer n * FactRec(n - 1)
    fin si;
FIN
```

mystere : cond(n = 0, 0, n*n + mystere(n−1)) = $1^2 + \dots + n^2 = n(n+1)(2n+1)/6$ · sommeInterv : cond(a = b, a, a + sommeInterv(a+1, b)) · suiteU : cond(n = 0, 1, 2*suiteU(n−1) + 3) · estPair : cond(n = 0, true, cond(n = 1, false, estPair(n−2))) · sommeImpairs : cond(n ≤ 0, 0, cond(n mod 2 = 1, n + s(n−2), s(n−1))) · mul (paysan russe) : a = 0 → 0 ; a pair → 2*mul(a div 2, b) ; sinon 2*mul(a div 2, b) + b.

Motifs récursifs : ligneRec(n) : si n = 0 : affRL() sinon affEtoile() ; ligneRec(n−1) · triDec(n) : si n > 0 : ligneRec(n) ; triDec(n−1) · rectangle(p, n) : si p > 0 : ligneRec(n) ; rectangle(p−1, n) · pyramideDec(n, p) : si n > 0 : pyramideDec(n−1, p+1) ; affLigneEsp(p) ; ligneRec(2n−1).

6. Récursivité : ce qui s'affiche

Repérer quand rien ne s'affiche ; calculer G(1), G(2), G(3) **dans l'ordre des instructions**, puis remplacer chaque appel ; écrire ces étapes sur la copie.

Sujet A : G(n−2) ; afficher n * ; G(n−1) → G(4) = ** / * / **** / * / *** / ** / *.
Sujet B : G(n−1) ; afficher ; G(n−2) → G(4) = * / ** / *** / * / **** / * / **.
PEIP : G(n div 2, d+1) puis n fois d → G(9, 0) = 3 / 22 / 1111 / 000000000.
Pingouin : trajet(3) = 0 1 0 2 0 1 3 = ← ↑ ← → ← ↑ ↓.

Arrêt (au cas où) : P_n : « G(n) s'arrête ». Init : n ≤ 0, seulement le test. Hérédité (à deux pas) : si P_{n-1}, P_{n-2}, les appels s'arrêtent, l'affichage aussi, donc P_n.

7. Tableaux 1D (indices 0 à taille(T) − 1)

Accumuler : init avant, mise à jour dedans, Renvoyer après. **Chercher** (tous / il existe) : Renvoyer dès qu'on sait, l'autre réponse après la boucle. **Fabriquer** : `Variable : R : tableau de taille(T) nombres`, remplir, Renvoyer R.

```
FONCTION : tousDiv3 (E T : tableau d'entiers) : booléen
SPÉCIFICATIONS : true si tous les éléments sont divisibles par 3.
DÉBUT
    pour i de 0 à taille(T)-1 faire
        si T[i] mod 3 ≠ 0 alors
            Renvoyer false
        fin si;
    fin pour;
    Renvoyer true
FIN
```

moyenne : som / taille(T) · inverse : R[i] ← T[taille(T)−1−i] · croissant : i de 0 à taille(T)−2, si T[i] > T[i+1] alors Renvoyer false · deuxiemeMin : m1, m2 (les deux plus petits), si T[i] < m1 alors m1, m2 ← T[i], m1 sinon si T[i] < m2 alors m2 ← T[i] · injectif (double tant que) : i, inj ← 0, true ; tant que inj et i < taille(T)−1 : j ← i+1 ; tant que inj et j < taille(T) : si T[i] = T[j] alors inj ← false · Volume (CC 2024) : V ← V + L[i]*W[i]*H[i].

8. Tableaux 2D, jeu de la vie

T[i][j] : ligne i, colonne j. Lignes : **taille(T)** ; colonnes : **taille(T[i])** (= taille(T[0])). Déclaration : `T : tableau l x c d'entiers`.

```
pour i de 0 à taille(T)-1 faire
    pour j de 0 à taille(T[i])-1 faire
        som ← som + T[i][j]
    fin pour;
fin pour;
```

TD 4 ex. 15 : som, prod, nb ← 0, 1, 0 ; dans la double boucle : som ← som + T[i][j] ; prod ← prod * T[i][j] ; nb ← nb + 1 ; puis som / nb et prod ^ (1/nb).
Dénivelé (ex. 16) : voisin de droite si j < taille(T[i])−1, du dessous si i < taille(T)−1 : d ← max(d, abs(T[i][j] − T[i][j+1])).

Voisines de (i, j) : a de i−1 à i+1, b de j−1 à j+1, `si a ≥ 0 et a < taille(G) et b ≥ 0 et b < taille(G[a]) et (a ≠ i ou b ≠ j) alors nb ← nb + G[a][b]` (le « et » paresseux protège les bords). **Règles** : vivante survit avec 2 ou 3 voisines ; morte naît avec exactement 3. Calculer dans un **nouveau** tableau H.

Pendu : vues tableau de booléens. propose(E c, E mot, ES vues) : parcourir **tout** le mot (lettre en double), vues[i] ← true, trouve ← true. gagne : false dès un « non vues[i] ».

9. Dictionnaires

`D : dictionnaire('taille' : nombre, 'age' : entier)` · `D['taille'] ← 168` · `pour k dans D faire … fin pour;` · `si k dans D alors`. compareDicos : chaque clé de D1 dans D2 avec la même valeur, puis aucune clé en plus dans D2.

10. Listes : tête, queue, cons, estVide

- tête(L) élément · queue(L) liste · cons(x, L) ajoute devant · estVide(L). **Pas de L[2]**. 3e élément : tête(queue(queue(L))).
- Un seul élément : estVide(queue(L)) · deux : estVide(queue(queue(L))).

```
Renvoyer cond(estVide(L), 0, 1 + longueurR(queue(L)))
Renvoyer cond(estVide(queue(L)), tête(L), max(tête(L), plusgrand(queue(L))))
Renvoyer cond(estVide(queue(queue(L))), tête(L), avantdernier(queue(L)))
Renvoyer cond(estVide(L), cons(x, [ ]), cons(tête(L), insereFin(queue(L), x)))
Renvoyer cond(n = 0, cons(0, [ ]), cons(n, listeEntiers(n - 1)))
```

Itératif (cours p. 40) : `cpt, M ← 0, L ; tant que non estVide(M) faire cpt ← cpt + 1 ; M ← queue(M) fin tq;`
listesEgales : cond(estVide(L1), estVide(L2), cond(estVide(L2), false, tête(L1) = tête(L2) et listesEgales(queue(L1), queue(L2)))) · prefixe : cond(estVide(l1), true, cond(estVide(l2), false, tête(l1) = tête(l2) et prefixe(queue(l1), queue(l2)))) · facteur : prefixe(l1, l2) ou (non estVide(l2) et facteur(l1, queue(l2))).
Tableau → liste dans l'ordre : `pour i de taille(T)-1 à 0 par pas de -1 faire L ← cons(T[i], L)`.
TD 5 ex. 17, li = [2, 3, 2, 4] : queue(queue(li)) = [2, 4] · tête(queue^4(li)) erreur · estVide(tête(li)) erreur · cons(tête(queue(li)), queue(li)) = [3, 3, 2, 4].

11. Piles et files

Pile : **empiler(e, P)**, **depiler(P)** (ES : enlève et renvoie le sommet), estVide(P). File : **enfiler(e, F)**, **defiler(F)** (le premier arrivé). Interdit : P[1], tête, queue, cons, empiler(P, 4).

Cours p. 44 : P ← [ ] ; empiler 1, 2, 3 → sommet 3 · empiler(depiler(P), P) : inchangé · empiler(P[1], P), empiler(P, 4), P ← cons(3, P) : **erreurs** · depiler(P) : reste 1, 2.
Vider une pile dans une autre **inverse** l'ordre : inverser P = 3 transvasements (P→Q, Q→R, R→P). doublePile : empiler(2*depiler(P), Q) puis tout remettre dans P. inverseFile : défiler dans une pile, puis dépiler dans la file.

12. Pseudo-code ↔ Python

x ← 3 : x = 3 · x = 3 (test) : x == 3 · ≠ : != · div : // · mod : % · ^ : ** · et ou non : and or not · cond(c, x, y) : x if c else y · pour i de a à b : for i in range(a, b+1) · pour i de n−1 à 0 par pas de −1 : range(n-1, -1, -1) · tant que : while · taille(T) : len(T) · Renvoyer : return.

Avant de rendre ta copie

- ☐ Signature complète : FONCTION/PROCÉDURE, E/S/ES, types, type du résultat (fonction seulement).
- ☐ Une ligne SPÉCIFICATIONS.
- ☐ Chaque variable déclarée, initialisée avant usage ; pas l'indice d'un pour.
- ☐ Chaque si, pour, tant que fermé : fin si; fin pour; fin tq;, aligné sous son ouverture.
- ☐ Dans un tant que, la variable du test change.
- ☐ Indices de 0 à taille(T) − 1.
- ☐ Renvoyer au bon endroit (après la boucle, sauf recherche).
- ☐ Récursif : un cas de base, un appel sur plus petit.